

JustEat

Autores: Martín Gutiérrez Bermejo y Raúl Hormigo Cerón

ÍNDICE

INTRODUCCIÓN..... 2
MODELO VISTA-CONTROLADOR..... 2
REQUISITOS..... 6
REQUISITOS MÍNIMOS.....6
REQUISITOS NO FUNCIONALES..... 7
SEGURIDAD..... 7
REQUISITOS FUNCIONALES.....7
REGISTRO Y ACCESO DE USUARIOS..... 7
GESTIÓN RESTAURANTES.....0
GESTIÓN DE PLATOS.....0
GESTIÓN DE PEDIDOS..... 0
VALORACIÓN..... 0
BUSCAR RESTAURANTE..... 0
ESTADO DEL RESTAURANTE.....0
REQUISITOS EXTRAS..... 0
GESTIÓN DE CATEGORÍAS..... 0
RESTAURANTES FAVORITOS..... 0
GESTIÓN DE IMÁGENES..... 0

INTRODUCCIÓN

Este proyecto trata de simular una aplicación web de reparto a domicilio como es Just Eat. Para esta tercera entrega se ha creado una aplicación que actúa como servicio en el cliente, de forma que no todo el trabajo lo realiza el servidor. Para ello, se ha empleado el mismo *server* de la entrega anterior pero añadiendo cierta funcionalidad en *JavaScript* utilizando *AngularJS*.

Para poder gestionar el proyecto correctamente, los miembros del grupo han utilizado el Sistema de Control de Versiones *Git* ofrecido por herramientas como *GitHub*. Además, se ha empleado una metodología ágil para desarrollar el proyecto al completo.

El presente documento se divide en varias secciones donde se relata tanto la arquitectura construida como las diferentes decisiones que se han tomado para el desarrollo del proyecto. Adicionalmente, se incluyen todas las funcionalidades que se ofrecen en la aplicación web para el uso y disfrute de los usuarios.

MODELO VISTA-CONTROLADOR

El Modelo-Vista-Controlador se trata del patrón de arquitectura propuesto por el profesorado y utilizado en el proyecto de la asignatura. Para comenzar se genera un modelo de Base de Datos que, si bien parte de lo otorgado por el profesorado de la asignatura; no es exactamente igual:

“La aplicación debe almacenar y mostrar la información de restaurantes. De cada uno de ellos se debe conocer su nombre, dirección, teléfono, valor medio de las valoraciones recibidas, ciudad en la que se ubica, precios mínimo y máximo que cobra a su clientela, e-mail de contacto, si cuenta con aparcamiento para bicicletas y si está disponible o no para recibir pedidos. Los restaurantes deben ser clasificados en diferentes categorías, ya sea en una o varias; pero no pueden existir categorías que no agrupen a ningún restaurante, aunque pueden agrupar a varios de ellos a la vez. De cada categoría se conoce su nombre y una descripción.

La aplicación debe controlar también a sus usuarios registrados. De cada usuario es necesario conocer su nombre, apellido, e-mail y contraseña con la que se registra. Los usuarios registrados deben ser propietarios de, al menos, uno o varios restaurantes; pero no pueden existir restaurantes que no estén asociados a una persona. Por otro lado, los usuarios pueden dejar valoraciones de otros restaurantes (no al suyo propio), y los restaurantes pueden contar con varias valoraciones o con ninguna. De cada valoración se debe conocer qué se ha indicado y la puntuación que ha recibido el restaurante.

Cada restaurante cuenta con varios platos que ofrece al público. De cada plato se debe mantener su nombre, precio y descripción; y no puede existir un plato que pertenezca a varios restaurantes a la vez.

Los usuarios pueden hacer pedidos a los restaurantes. Los usuarios pueden hacer cuantos pedidos quieran (o ninguno), pero los pedidos sólo pueden ser solicitados por un usuario. Los pedidos deben contar con cuantos platos se deseen, pero no todos los platos tienen por qué ser solicitados en un pedido.”

Una vez claras todas las características y restricciones con las que se cuenta, se genera el siguiente modelo Entidad-Relación que resume lo explicado anteriormente:

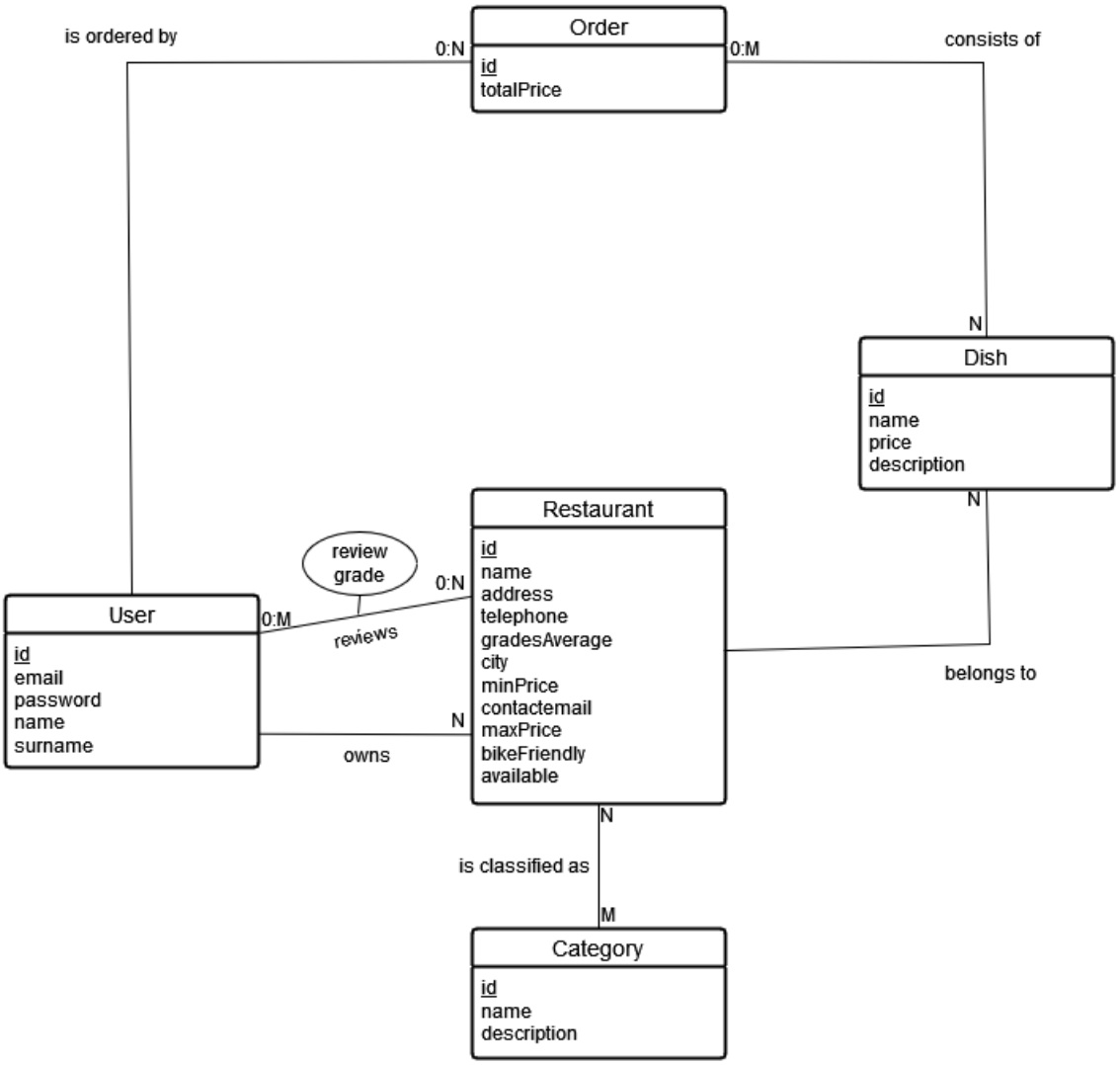


Figura 1. Modelo Entidad-Relación que ilustra el modelo de datos redactado anteriormente.

Cabe destacar que la única restricción no representada con este modelo es "*Un usuario no puede dejar una valoración a su propio restaurante*", por lo que debe ser gestionada más adelante durante la creación de la aplicación.

El Modelo Entidad-Relación debe convertirse en un Modelo Relacional que permita representar fácilmente el modelo de datos en un esquema relacional de Base de Datos. De esta forma, el Modelo Relacional consta de:

RESTAURANT (<u>id</u>, name, address, telephone, gradesAverage, city, minPrice, contactEmail, maxPrice, bikeFriendly, available, idu) FK = {idu REFERENCES User (id) ON DELETE CASCADE}
CATEGORY (<u>id</u>, name, description)
RESTAURANT_CATEGORY (<u>idr, idct</u>) FK = {idr REFERENCES Restaurant (id) ON DELETE CASCADE} FK = {idct REFERENCES Category (id) ON DELETE CASCADE}

```
USER (id, email, password, name, surname)
```

```
RESTAURANT_USER (idr, idu, review, grade)
    FK = {idr REFERENCES Restaurant (id) ON DELETE CASCADE}
    FK = {idu REFERENCES User (id) ON DELETE CASCADE}
```

```
DISH (id, name, price, description, idr)
    FK = {idr REFERENCES Restaurant (id) ON DELETE CASCADE}
```

```
ORDER (id, idu, totalPrice)
    FK = {idu REFERENCES User (id) ON DELETE CASCADE}
```

```
ORDER_DISH (ido, iddi)
    FK = {ido REFERENCES Order (id) ON DELETE CASCADE}
    FK = {iddi REFERENCES Dish (id) ON DELETE CASCADE}
```

Figura 2. Modelo Relacional del modelo de datos generado a partir del Modelo Entidad-Relación expuesto en la *Figura 1*.

Una vez determinado el Modelo Relacional que se aplicará para generar la Base de Datos, es necesario especificar los tipos de datos de cada atributo dentro de cada relación. Para ello, se muestran las sentencias SQL que crean cada tabla:

```
CREATE TABLE "Restaurant" (
    "id"    INTEGER NOT NULL,
    "name"    TEXT,
    "address"    TEXT,
    "telephone"    TEXT,
    "gradesAverage"    REAL,
    "city"    TEXT,
    "minPrice"    INTEGER,
    "contactEmail"    TEXT,
    "maxPrice"    INTEGER,
    "bikeFriendly"    INTEGER,
    "available"    INTEGER,
    "idu"    INTEGER NOT NULL,
    PRIMARY KEY("id" AUTOINCREMENT),
    FOREIGN KEY("idu") REFERENCES "User"("id") on delete cascade
);
```

```
CREATE TABLE "Category" (
    "id"    INTEGER NOT NULL,
    "name"    TEXT,
    "description"    TEXT,
    PRIMARY KEY("id" AUTOINCREMENT)
);
```

```
CREATE TABLE "Restaurant_Category" (
    "idr"    INTEGER NOT NULL,
    "idct"    INTEGER NOT NULL,
    FOREIGN KEY("idct") REFERENCES "Category"("id") on delete cascade,
    FOREIGN KEY("idr") REFERENCES "Restaurant"("id") on delete cascade,
    PRIMARY KEY("idr","idct")
);
```

```
CREATE TABLE "User" (
    "id"    INTEGER NOT NULL,
    "email"    TEXT,
    "password"    TEXT,
    "name"    TEXT,
    "surname"    TEXT,
    PRIMARY KEY("id" AUTOINCREMENT)
);
```

```
CREATE TABLE "Restaurant_User" (
    "idr"    INTEGER NOT NULL,
    "idu"    INTEGER NOT NULL,
    "review"    TEXT,
    "grade"    REAL,
    PRIMARY KEY("idr","idu"),
    FOREIGN KEY("idr") REFERENCES "Restaurant"("id") on delete cascade,
    FOREIGN KEY("idu") REFERENCES "User"("id") on delete cascade
);
```

```
CREATE TABLE "Dish" (
    "id"    INTEGER NOT NULL,
    "name"    TEXT,
    "price"    INTEGER,
    "description"    TEXT,
    "idr"    INTEGER NOT NULL,
    PRIMARY KEY("id" AUTOINCREMENT),
    FOREIGN KEY("idr") REFERENCES "Restaurant"("id") on delete cascade
);
```

```
CREATE TABLE "Order" (
    "id"    INTEGER NOT NULL,
    "totalPrice"    INTEGER,
    "idu"    INTEGER NOT NULL,
    FOREIGN KEY("idu") REFERENCES "User"("id") on delete cascade,
    PRIMARY KEY("id" AUTOINCREMENT)
);
```

```
CREATE TABLE "Order_Dish" (
    "ido"    INTEGER NOT NULL,
    "iddi"    INTEGER NOT NULL,
    FOREIGN KEY("ido") REFERENCES "Order"("id") on delete cascade,
    FOREIGN KEY("iddi") REFERENCES "Dish"("id") on delete cascade,
    PRIMARY KEY("ido","iddi")
);
```

Figura 3. Código SQL que genera las relaciones descritas en el Modelo Relacional de la *Figura 2*.

Una vez construido todo el Modelo de Datos, es necesario generar y adaptar el patrón *Data Access Object* para poder mapear las relaciones que constituyen la Base de Datos con los objetos Java que se utilizarán en el *server* de la aplicación. En primer lugar, es crucial establecer una relación biyectiva entre los tipos de datos que soporta el *DBMS* y los tipos de datos soportados en Java:

```
INTEGER = int
TEXTO = String
REAL = double
```

Figura 3: Mapeo entre los tipos de datos del DBMS a Java

Tras esto, se crean los *POJO*'s básicos que representan las relaciones de la Base de Datos en el contexto del *server*. Cabe destacar que no se aplica ningún tipo de mapeo entre ellos o algún otro ORM existente; por lo que la conversión es directa. Esto es lo que explica la existencia de los archivos en el paquete "*es.unex.pi.model*" y el que sus nombres y atributos sean igual a la tabla que representan. Por otro lado, se han generado interfaces que ofrecen los servicios necesarios de la Base de Datos, así como sus implementaciones. Este código es que comunica el *server* con el DBMS y permite independizarlo de forma efectiva.

Existe una diferencia fundamental entre el código ofrecido por el profesorado de la asignatura y el entregado por los estudiantes: la comunicación con el *DBMS* se realiza utilizando las clases "*Statement*" de JDBC, mientras que los estudiantes han optado por utilizar "*PreparedStatement*". De esta forma se aumenta la seguridad y fiabilidad de la comunicación y ofrece una mayor protección ante, por ejemplo, inyecciones SQL.

REQUISITOS

En esta segunda fase del proyecto se formulan una serie de requisitos clasificados en mínimos y extra. Los primeros son, a su vez, divididos en requisitos funcionales y no funcionales; y suponen un 60% de la nota total del proyecto. Adicionalmente, se proponen varios requisitos adicionales que permiten subir la nota del proyecto hasta un máximo de 10.

REQUISITOS MÍNIMOS

Los requisitos mínimos, como se ha comentado, se han dividido en no funcionales y en aquellos que suponen un aumento de los servicios ofrecidos.

REQUISITOS NO FUNCIONALES

SEGURIDAD

Para proporcionar una buena seguridad en la página web se han construido cinco filtros diferentes que evitan que ciertos usuarios y no registrados puedan acceder a zonas no autorizadas.

El filtro del inicio de sesión no permite a aquellos usuarios no *loggeados* acceder a las pantallas que constituyen la aplicación, incluso aunque tengan el enlace directo que les dirige a ellas. De esta forma se protege la información de los usuarios que están registrados en la aplicación.

El filtro de la gestión de restaurantes ofrece protección real contra el acceso a las pantallas de manejo de los restaurantes de un usuario. En este sentido, se asegura de que el cliente es propietario del restaurante que está editando; no permitiendo el acceso a sus datos en caso de que no lo sea. En los datos de un restaurante se incluyen los platos que ofrece que, a pesar de estar protegido por un filtro diferente, supone el mismo nivel de protección que los restaurantes en cuando al visionado y edición de los datos se refiere.

La pantalla de pedidos de un usuario protege la información de los clientes frente a aquellas personas que no han solicitado dichos pedidos. En este sentido, sólo deja gestionarlo a aquellos clientes que sí han realizado el pedido al restaurante indicado. Adicionalmente, se trata de un sistema de protección muy similar al que maneja las valoraciones de los usuarios a los restaurantes; dejando solamente a aquellos usuarios propietarios de las valoraciones editar la información de las mismas; protegiendo de esta forma los datos entregados.

REQUISITOS FUNCIONALES

REGISTRO Y ACCESO DE USUARIOS

El registro de usuarios se realiza con un formulario que recoge sus datos personales. Tras ello, se asegura que la información otorgada cumple ciertas condiciones (como que la contraseña es robusta frente a ataques). Todos los datos redactados se almacenan en la Base de Datos con un identificador de usuario único. Por otro lado, los clientes tienen la posibilidad de eliminar su información almacenada en cualquier momento desde su cuenta; además de poder visualizarla y editarla.

Por otro lado, existe la posibilidad de iniciar sesión en la aplicación proporcionando el correo electrónico y la contraseña asociadas al usuario (ya que son únicos para cada persona). También se da la opción de cerrar sesión en la aplicación web desde la pantalla de cuenta del usuario.

GESTIÓN RESTAURANTES

La gestión de restaurantes supone la implementación de las cuatro operaciones básicas que se deben ofrecer: creación, lectura, actualización y borrado de la información. De esta forma, cualquier usuario registrado en la aplicación puede crear un restaurante indicando datos como son su nombre de negocio, dirección, teléfono, correo electrónico, tipo de comida que sirve, si cuenta con aparcamiento para bicicletas, ciudad en la que se sitúa, si acepta pedidos, etc. Existe información adicional sobre los restaurantes como son la media de las valoraciones recibidas o los precios mínimo y máximo que ofrece; pero son calculados en tiempo real durante el procesamiento de los datos.

Además, un propietario de restaurante puede editar la información del mismo en cualquier momento desde la pantalla de edición (exceptuando aquellos valores calculados), además de tiene opción de eliminarlo si lo desea.

GESTIÓN DE PLATOS

La gestión de platos se accede desde la pantalla de edición de restaurante, donde se puede realizar las cuatro operaciones básicas de cualquier tipo de información. De forma análoga a los establecimientos, los platos pueden ser creados y asociados automáticamente a su restaurante, su información puede ser editada en cualquier momento y puede ser eliminada a voluntad. Toda la información de los platos se puede observar en la pantalla de detalles de un restaurante. De cada plato se recoge información como su nombre, precio y descripción.

GESTIÓN DE PEDIDOS

La gestión de pedidos se resume a su solicitud a los restaurantes (desde su pantalla de detalles) y a su revisión si se desea. Ya que se debe mantener un historial de todos los pedidos realizados, no se permite el borrado o la edición de la información proporcionada por los mismos. Todo este manejo de los pedidos se realiza desde la pantalla de pedidos del usuario, donde éste puede observar todos las solicitudes que ha hecho a los restaurantes, así como su precio total de compra.

Cabe destacar que los pedidos se clasifican por el restaurante al que se ha realizado, por lo que sólo se pueden hacer desde la pantalla de detalles del establecimiento; y eligiendo sobre todos los platos que tienen disponibles.

VALORACIÓN

Las valoraciones se pueden realizar desde la pantalla de detalles de cada restaurante. Cabe destacar que solo se puede realizar una por usuario, además de indicar un comentario sobre el servicio. La aplicación web permite actualizar la información ofrecida y eliminarla si es necesario. También muestra el nombre de los usuarios que realizaron la valoración.

En su pantalla principal, la aplicación web permite ordenar todos los restaurantes existentes en función de la media de las valoraciones ofrecidas; mostrando primero aquellos que están mejor calificados por los clientes.

BUSCAR RESTAURANTE

La aplicación permite realizar un filtrado de los restaurantes en función de parámetros como son su nombre, ciudad o calle en la que se ubican. Resulta una funcionalidad básica y principal en la aplicación.

ESTADO DEL RESTAURANTE

En la pantalla principal se muestra un listado con todos los restaurantes almacenados en la aplicación. Sobre esta se puede aplicar un filtro que permita discriminar entre aquellos restaurantes que no se encuentran disponibles para recibir pedidos y aquellos que no lo están; recargando la página para mostrar la nueva información al usuario.

REQUISITOS EXTRAS

GESTIÓN DE CATEGORÍAS

A la hora de crear o editar la información almacenada de un restaurante, los propietarios de los mismos tienen la posibilidad de clasificarlo en una serie de categorías definidas. De esta forma, un restaurante puede clasificarse entre una o varias categorías al mismo tiempo en la aplicación y sin que exista un límite definido por la misma. Adicionalmente, en la sección de estados se permite clasificar los restaurantes por las categorías existentes en la aplicación, dando así posibilidad al usuario de realizar una mejor búsqueda de sus restaurantes favoritos.

RESTAURANTES FAVORITOS

En la sección de estados se permite filtrar por aquellos restaurantes que el usuario ha asignado como favoritos. De esta forma, en los detalles del mismo se puede decidir si un restaurante se marca como favorito o no. Tal y como expresa en el enunciado, sólo se puede marcar un restaurante como favorito una única vez. Para implementar este requisito se ha modificado la Base de Datos y añadido una nueva relación que recoge esta funcionalidad.

GESTIÓN DE IMÁGENES

En las operaciones C.R.U.D. de los restaurantes se ha añadido la posibilidad de asociarles una imagen que los identifique (siempre que ya esté guardada en el servidor). Para ello, se ha modificado la Base de Datos y permite guardar su localización en el sistema.